Date: 20/10/2026

SAI DATTA BHAAVAN MBNP
241401087 B.Tech CSBS

APPLYING PERFORMANCE OPTIMIZATION TECHNIQUES IN ZOHO CREATOR

Course: PE - Low-Code Application Development using Zoho Creator

Application under study: Loan Application and Pre-screening Management

Form name: Loan_Application_Form **Report name:** Loan_Application_Form Report

AIM

To study and apply practical performance-oriented configuration techniques in a Zoho Creator application by using report filters to narrow visible records, sorting records in a useful order, and organizing repeated liability information inside a subform. The experiment also examines the availability of manual indexing controls in the current interface and records the limits of what could be verified.

1. INTRODUCTION

Low-code platforms make it possible to build forms, reports, workflows, and dashboards without writing a complete application from scratch. As the number of records and the complexity of a form increase, however, the way information is organized and retrieved becomes important. A report that displays every application may be harder for an officer to review than a report restricted to one loan category. Related information, such as multiple existing liabilities for one applicant, should be grouped in a consistent structure rather than placed into a long collection of unrelated fields.

Performance optimization includes both actual execution improvements and practical design changes that make the application easier to use. Indexing can improve lookup operations when a platform supports configurable indexes. Query tuning reduces unnecessary work by applying suitable criteria. Sorting makes records easier to scan, while subforms model one-to-many relationships in a structured way. These techniques have different purposes and should not be treated as interchangeable.

241401087

2. OBJECTIVES

1. Configure and verify a report filter using the Loan_Type field.
2. Configure ascending sorting by Applicant_Name and inspect the resulting report order.
3. Configure a subform named Existing_Liabilities_Details with Liability_Type, Outstanding_Amount, and Monthly_Payment fields.
4. Submit a loan application containing liability details and check that the related details are retained with the parent record.
5. Inspect the form and field settings for a manual indexing control, and document the observed result without assuming an index was created.

3. APPLICATION CONTEXT

The application is designed for collecting loan enquiries and supporting preliminary review by bank staff. The parent form stores applicant details, employment and income information, loan type, purpose, and consent. Existing liabilities are related details that may include more than one item per applicant. The associated report is used to review submitted applications and to filter or sort the list for a particular screening task.

4. BACKGROUND AND TECHNICAL CONCEPTS

4.1 Report filtering and query narrowing

A filter specifies which records should be included in a report view. In this experiment, the criterion is Loan_Type equals Home Loan. When the report is viewed with that criterion, records whose loan type does not match are excluded from the displayed result. Filtering is useful for a loan officer who wants to focus on one category rather than review all application types at once.

Filtering the displayed records demonstrates correct report configuration and can reduce the number of rows a person needs to inspect. It does not, by itself, prove that the database query ran faster: the platform may manage query execution internally, and no timed benchmark or execution plan was captured for this experiment.

4.2 Sorting

Sorting orders records by a selected field. The report was configured to sort by Applicant_Name in ascending order. A stable, predictable order improves readability, makes records easier to scan, and supports manual comparison. Sorting is principally an organization feature in this exercise; no claim is made that it independently reduced execution time.

4.3 Subforms and related data

A subform is useful when one parent record can have several related child entries. For example, one loan applicant may have a personal loan, a credit-card balance, or another monthly commitment. Storing each liability as a separate row in Existing_Liabilities_Details allows each type to have its own outstanding amount and monthly payment. This structure is more scalable and clearer than creating fixed fields such as Liability_1, Liability_2, and Liability_3 on the main form.

4.4 Indexing

An index is a data structure that can help a database locate matching records without inspecting every record in the same way as a full scan. Indexes are often useful on fields frequently used in search, filtering, or joins. They are not universally beneficial: they can take storage and may add overhead to changes. Whether a low-code application builder exposes manual index controls depends on its product features, field type, account, and interface. This experiment records only the setting exposed in the current session.

241401087

5. OPENING THE EXISTING APPLICATION

The existing application was opened from the Zoho Creator workspace. The report area was selected to access Loan_Application_Form Report and its configuration panel. The same application was used throughout so that the filter, sort order, and subform work related to the existing loan workflow rather than a newly created example.

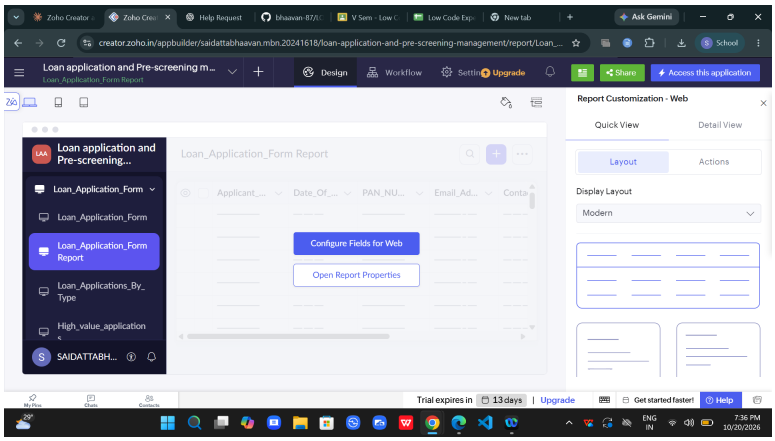


Fig. 1: Existing Loan Application and Pre-screening Management application, with the report area selected.

6. TECHNIQUE I - REPORT FILTERING / QUERY NARROWING

6.1 Purpose

The purpose of the filter is to display only the loan applications relevant to one screening category. Bank staff may need to review Home Loan applications separately from Personal Loan or other categories. Filtering by a meaningful field reduces visual clutter and allows a reviewer to concentrate on a subset of the report.

6.2 Configuration procedure

1. Open the existing app and select *Loan_Application_Form Report*.
2. Open the report in edit or builder mode.
3. Select *Open Report Properties* and then open the *Filter* section.
4. Click *+ Add Filter* to create a condition.
5. Choose *Loan_Type* as the field, *equals* as the operator, and *Home Loan* as the value.
6. Save the filter configuration and click *Done* to return to the report.
7. Open or refresh the report and verify the records displayed against the selected category.

6.3 Filter condition

Property	Configured value	Meaning
Field	Loan_Type	The report checks the application loan category.
Operator	equals	Only records with an exact matching value satisfy the condition.
Value	Home Loan	The selected category used for this test.

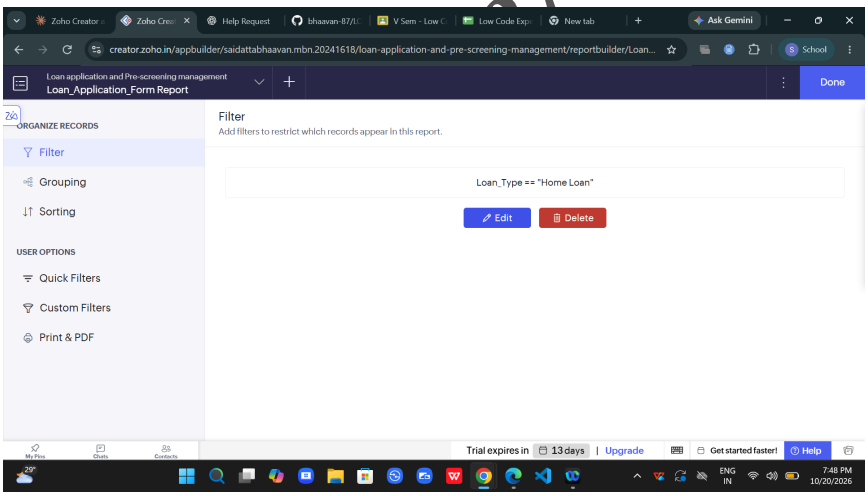


Fig. 2: Filter properties showing the Loan_Type condition for Home Loan.

6.4 Observation and interpretation

After the condition was saved, the report displayed records matching Home Loan. This confirmed that the report-level filter was active and behaving as configured. The configuration also makes the report more task-specific: a user reviewing Home Loan enquiries does not need to visually separate those entries from other loan types.

Measurement boundary: the successful result is evidence of correct filtering. No execution-time comparison was recorded before and after the filter, so this observation should not be reported as a measured speed improvement.

7. TECHNIQUE II - SORTING REPORT RECORDS

7.1 Purpose of sorting

Reports are easier to inspect when entries appear in a consistent order. Sorting the report by Applicant_Name in ascending order helps staff scan the list alphabetically, locate a known applicant, and compare entries without the list changing arbitrarily between views. Sorting is particularly convenient when there are several submitted applications.

7.2 Configuration procedure

1. Open *Loan_Application_Form Report* in edit mode.
2. Open *Report Properties* and choose *Sorting*.
3. Add or select *Applicant_Name* as the sort field.
4. Select *Ascending* order.
5. Save the report settings and return to the report view.
6. Inspect the visible applicant names to confirm that the report is ordered alphabetically.

7.3 Configuration evidence

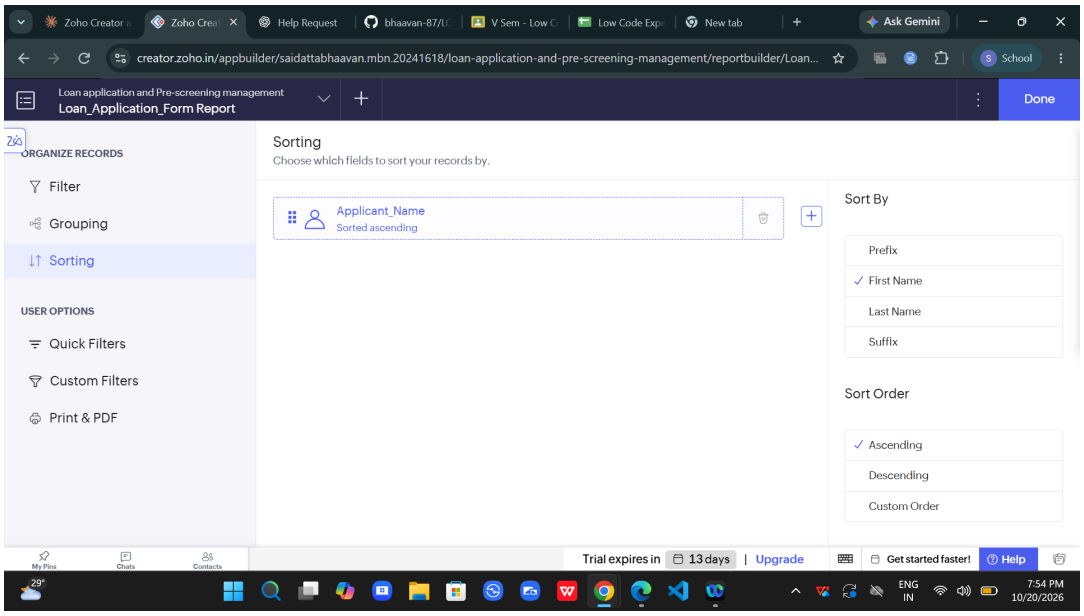


Fig. 3: Report Sorting settings showing Applicant_Name with ascending order selected.

7.4 Result and practical value

The sorting configuration was saved and the report was inspected. The visible records appeared in alphabetical order by applicant name, supporting the intended configuration. Ascending order is a useful default for review work because a reviewer can move through the list from A to Z. A different task might use descending date order to show the newest applications first.

Sorting does not remove records from the report; it changes their order. Filtering and sorting therefore serve separate purposes. A filter determines which rows are included, while sorting determines the order in which the included rows are presented.

7.5 Distinction between usability and performance

This experiment verifies the sort order rather than profiling the database. For a large data set, visible response time can depend on the platform, network, report complexity, and server-side implementation. A controlled performance study would time the same operation under comparable conditions and repeat the measurement. Such timing was not part of this practical session.

8. TECHNIQUE III - SUBFORM DESIGN FOR EXISTING LIABILITIES

8.1 Why a subform is suitable

A borrower may have zero, one, or several existing liabilities. The number of liability entries is therefore variable. A subform supports this one-to-many pattern by keeping multiple related rows under the parent loan application. Each row can contain the liability category, outstanding balance, and monthly payment, allowing the information to be reviewed consistently.

This design avoids an unnecessary fixed limit on the number of liabilities represented directly on the parent form. It also avoids repeating separate fields for each possible liability. Structured child rows are easier to validate, maintain, and understand in the context of the associated application.

8.2 Fields created in the subform

Field name	Purpose	Example of expected value
Liability_Type	Identifies the type of financial commitment.	Personal Loan, Credit Card, Car Loan
Outstanding_Amount	Stores the outstanding balance for that liability.	50000
Monthly_Payment	Stores the recurring monthly payment.	5000

8.3 Configuration procedure

1. Open *Loan_Application_Form* in the Form Builder.
2. Choose the Subform component from the field palette and add it to the form layout.
3. Rename the subform *Existing_Liabilities_Details* so its purpose is clear.
4. Add the fields *Liability_Type*, *Outstanding_Amount*, and *Monthly_Payment* inside the subform.
5. Review the field names and types to make sure the amount fields accept numeric or currency-style values suitable for financial entries.
6. Save the form changes before testing it in the live form.

241401087

8.4 Builder evidence

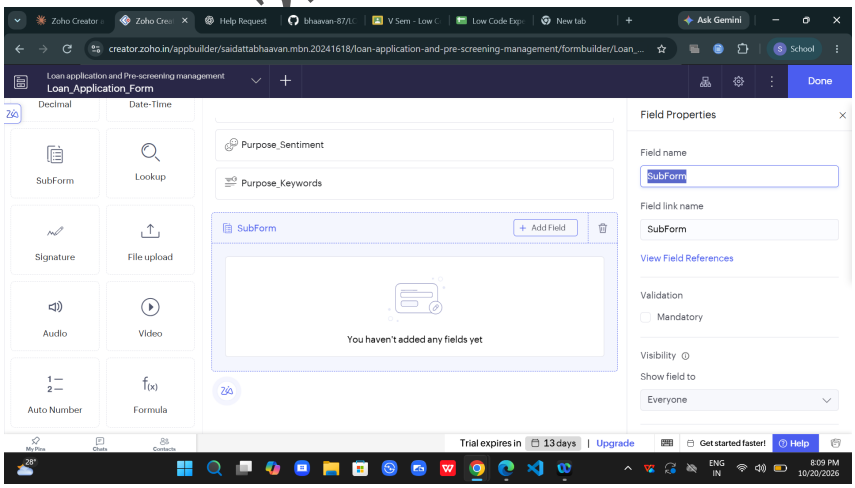


Fig. 4: Form Builder screen during the creation of the Existing_Liabilities_Details subform.

The screenshot records the subform component in the form builder. The subform itself is the container for the repeated related fields; the main loan application remains the parent record.

9. SUBFORM FIELD CONFIGURATION AND TESTING

9.1 Field configuration

After adding the subform container, the child fields were configured within it. Liability_Type identifies the commitment, Outstanding_Amount captures the remaining amount owed, and Monthly_Payment records the repayment amount due each month. These three values give a reviewer a compact view of a liability and support later screening or reporting.

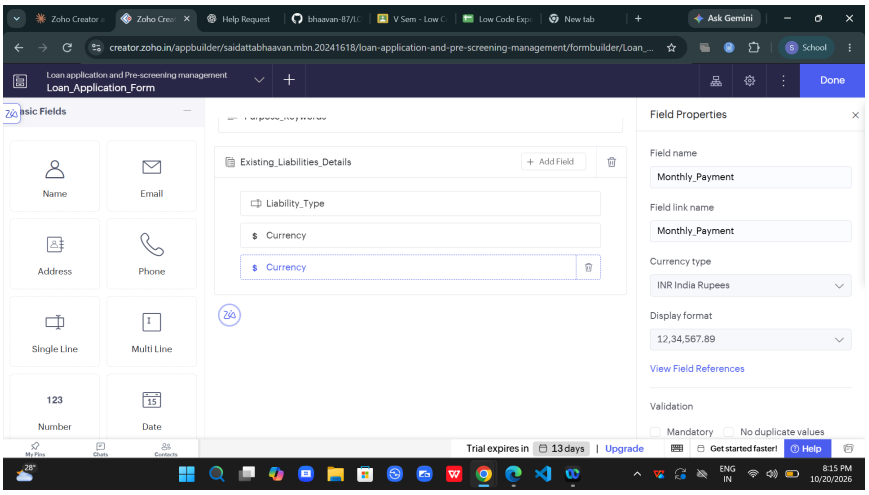


Fig. 5: Subform fields shown while configuring the liability details.

9.2 Live-form test procedure

1. Open the live form or access mode for the loan application.
2. Complete the required parent application fields using suitable test data.
3. In Existing_Liabilities_Details, click + *Add New* to add a liability row.
4. Enter a liability type, outstanding amount, and monthly payment. For example, a test entry may use Personal Loan, 50000, and 5000.
5. Submit the complete application.
6. Open the newly submitted record and inspect the liability details to verify that the subform data is retained with the parent application.

241401087

9.3 Live-form evidence

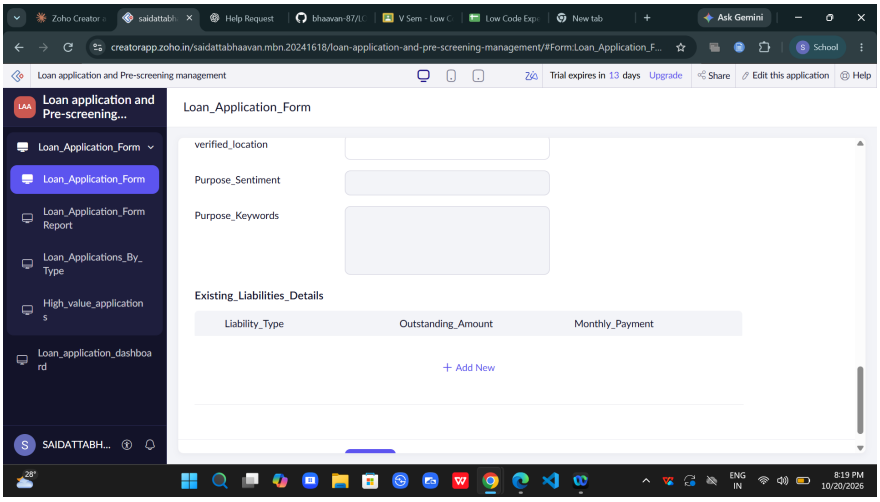


Fig. 6: Live loan application displaying the Existing_Liabilities_Details subform and its row-entry area.

9. SUBFORM VERIFICATION, QUALITY CHECKS, AND OBSERVATIONS

9.4 Observation from the submitted application

A new loan application was submitted with subform details. The submitted record was then checked, and the liability information was visible with the newly added application. This confirms that the form captures related liability values as part of the application flow. It demonstrates correct structure and data capture; it is not a direct benchmark of runtime performance.

9.5 Data-quality considerations

In a production financial application, amount fields should accept only appropriate positive values, and users should be able to distinguish the outstanding balance from the recurring monthly payment. Clear field labels, validation rules, and consistent numeric formats reduce errors. Access control should also limit who can view sensitive financial data. These are design recommendations; they should be tested as separate validation rules if they are to be enforced in the app.

9.6 Suggested verification cases

Test case	What to check	Expected / recommended outcome
Single liability row	Add one liability type with an amount and monthly payment.	One child row should be associated with the correct loan application.
Multiple liability rows	Use the subform Add New action to enter more than one liability.	Each liability should remain a separate row rather than overwriting the prior entry.
Numeric values	Enter realistic outstanding and recurring payment values.	The fields should display values consistently; positive-value validation is recommended.
Parent-record check	Submit and reopen the loan application.	The parent application and its child liability entries should remain associated.
Privacy review	Inspect which application users can view the saved financial data.	Permissions should be limited to the roles that need access.

241401087

The table describes recommended test cases for a complete validation pass. The performed checks confirmed that a submitted application retained its entered liability details. The table should not be read as proof that every recommended edge case was individually executed during this session.

9.7 Relation between parent form and subform

The parent loan form continues to hold the main applicant and loan details, while Existing_Liabilities_Details holds the repeated child entries. This separation gives each part of the application a clear responsibility: parent fields describe the applicant and requested loan, and subform rows describe existing commitments. Consistent structure helps future report design and maintenance.

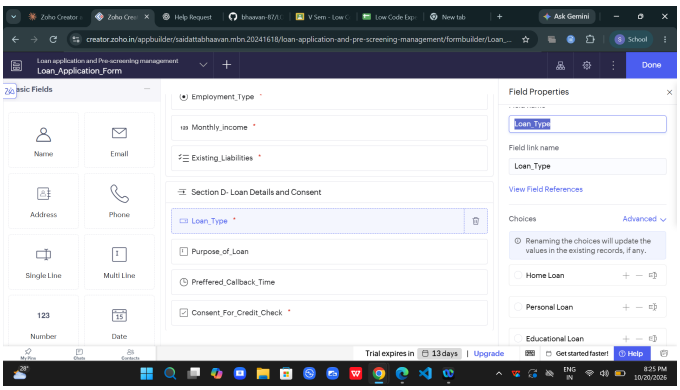


Fig. 7: Parent form builder view showing the loan details section and the Loan_Type and Purpose_of_Loan fields.

10. INDEXING: INSPECTION AND LIMITATIONS

10.1 Why indexing was considered

Indexing is a database-level technique often used to support faster searching and filtering on selected fields. In the context of a loan application, fields frequently used to retrieve records, such as a category or a unique identifier, may be candidates for indexing when the platform provides this control. Indexing decisions should be based on common query patterns and supported platform capabilities rather than applied indiscriminately.

10.2 Settings inspected

During the session, the Loan_Type field properties were opened and inspected. The Form Properties menu was also opened. The visible controls did not include a manual Index option. The field menu did not expose an indexing command either. Based on what was visible in the interface examined, the practical record therefore marks manual indexing as **not configured / not available in the inspected UI**.

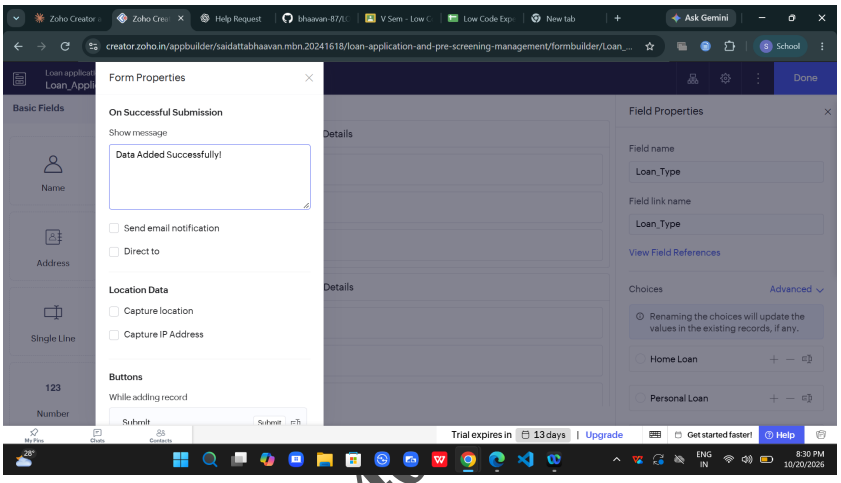


Fig. 8: Form Properties menu inspected while looking for a manual indexing control.

10.3 Important qualification

The absence of a visible manual index control does not establish that Zoho Creator has no internal indexing or performance management. It only means that a user-accessible option was not found in the specific field and form settings inspected in this session. The experiment does not claim that an index was created or enabled.

10.4 What would be required to verify index performance

A stronger indexing experiment would require an exposed index setting or documented platform support, a repeatable set of records, and a reproducible search operation. Measurements should be collected before and after the change under the same conditions. If the platform does not expose indexes, a practical report should state this limitation and focus on supported configuration options rather than simulating or inventing a setting.

10.5 Recommended next step for a deployment

For a production deployment, consult the product documentation or administrator interface available to the account to determine whether indexing is managed automatically or can be controlled through a supported feature. Avoid adding an unsupported custom workaround for financial data retrieval. This lab records the observed interface and does not infer undocumented backend behavior.

11. OBSERVATIONS AND TEST RECORD

The following table summarizes the configuration actions and the evidence observed during the working session. It separates verified outcomes from areas that were inspected but not directly configurable or measured.

Technique / item	Action performed	Observed outcome	What the evidence supports
Report filtering	Set Loan_Type equals Home Loan.	Report displayed matching Home Loan records.	Filter condition is saved and working.
Sorting	Set Applicant_Name to ascending order.	Visible names were arranged alphabetically.	Sort configuration is working for the report.
Subform design	Added Existing_Liabilities_Details and three child fields.	Subform and child-field setup visible in the builder.	Related liability data is modeled as repeated child rows.
Subform submission	Submitted a parent application with liability details.	Liability values appeared with the submitted application.	The user-entered subform data was retained with the parent record.
Manual indexing	Inspected field and form settings.	No manual index command was visible in the settings checked.	Indexing cannot be claimed as configured from this session.
Performance timing	No before-and-after benchmark performed.	No time measurements or execution plan were captured.	No numeric speed improvement can be claimed.

241401087

11.1 Report data view

The existing report was also opened to inspect submitted records and available fields. Reports provide the practical verification point after a form is submitted: they show parent application details and allow the configured view to be reviewed. The report view complements the builder screenshots by confirming that the app contains records rather than only an unsaved design.

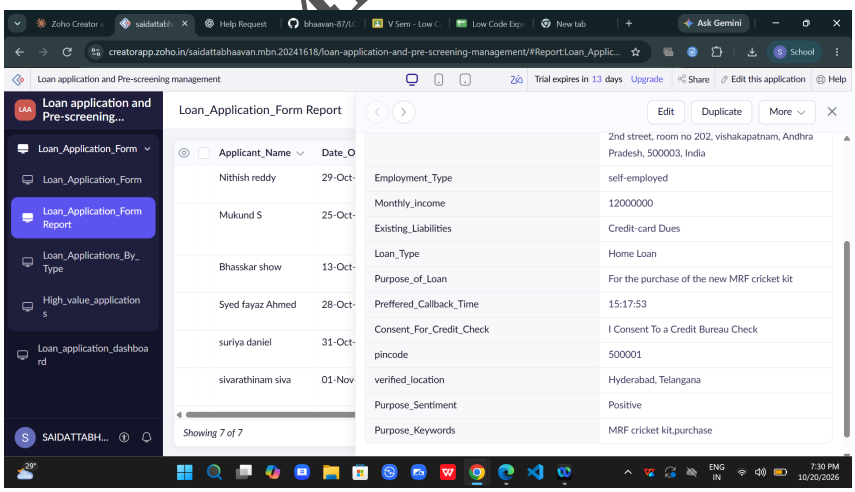


Fig. 9: Existing report record view used to inspect saved application data.

11.2 Summary of the test sequence

The test sequence was: configure report filter; verify matching records; configure ascending sort; inspect the sorted report; add the liability subform and child fields; submit a loan application with test liability values; and reopen the saved application to confirm the subform details. Form settings were reviewed for an indexing control, but none was found in the examined menus.

12. DISCUSSION AND PRACTICAL RECOMMENDATIONS

12.1 Use filters that match the user task

A report should show the fields and records needed for a specific job. A loan officer who is reviewing one category of loans can use a saved filter rather than repeatedly scanning the full report. When multiple criteria are needed, each condition should be checked carefully so that the report does not exclude valid applications unintentionally. Any saved filter should be labelled or documented clearly if it becomes part of routine work.

12.2 Keep sorting predictable

Alphabetical sorting by Applicant_Name is convenient for finding a known person. For operational queues, date-based sorting may be better when the most recent applications need attention first. The sort field and direction should reflect the review task. Sorting improves navigation, but it should not be described as query acceleration unless a performance test demonstrates that effect.

12.3 Model repeatable related data with a subform

The liability subform represents a variable number of commitments and keeps the data close to its parent application. Consistent field names and appropriate field types make the data easier to maintain. When the application grows, developers should test how child-row fields appear in record details and in reports, and should make sure that required values and numeric limits are validated.

12.4 Measure performance rather than assume it

A useful benchmark compares the same task before and after a supported configuration change. The data volume, filter condition, browser, network, and test procedure should stay as similar as possible. Multiple runs can reduce the impact of temporary delays. Record the timings and report the method. In this experiment, no measured before-and-after timing was taken; the outcomes are functional verifications of filter, sort, and subform behavior.

241401087

13. RESULT

Report filtering was configured using Loan_Type equals Home Loan and the report showed matching records. Sorting by Applicant_Name in ascending order was configured and the visible report order was checked. The Existing_Liabilities_Details subform was configured with Liability_Type, Outstanding_Amount, and Monthly_Payment, and submitted liability details were verified with the parent loan application. A manual indexing option was not visible in the inspected field/form settings, so indexing is documented as not configured in this session. No quantitative performance benchmark was performed.

14. CONCLUSION

The experiment demonstrated practical report filtering, sorting, and structured capture of repeated liability data in the Zoho Creator loan application. These configurations improve the usefulness and maintainability of the application by helping users focus on relevant records, review records in a predictable order, and store related financial commitments in a consistent subform. The indexing portion was limited by the absence of a visible manual indexing control in the settings examined. The results therefore distinguish verified application behavior from indexing or speed gains that were not established.

15. LIMITATIONS

The screenshots document the configuration and functional checks performed during this session. They do not provide a database execution plan, backend index details, or numerical response-time measurements. The result should be interpreted as a practical low-code configuration exercise rather than a controlled database performance benchmark.

Submitted by: SAI DATTA BHAAVAN MBNP **Roll No.:** 241401087 **Programme:** B.Tech CSBS